

Software Developer — Technical Test

Congratulations, you've been shortlisted for the technical test.

This is a two-part technical exercise for our Software Developer role. It is designed to reflect the work you will actually do on the job — debugging a real piece of our production code, and building a small UI component using AI-assisted tooling.

What we're evaluating

You are applying for an AI-native engineering role. Day-to-day, we build and ship features using **Claude Code CLI**, **Codex**, and **Figma MCP**. This test is designed to see:

- Whether you can read unfamiliar code and find real bugs
- Whether you can use agentic AI tools (Claude Code CLI / Codex) and Figma MCP effectively
- Whether you verify AI output or trust it blindly
- Whether your commit history, comments, and communication are clean enough for a team environment

AI tools are encouraged across all parts. Please do not hide your AI usage — we want to see it. You must, however, be able to explain every line of code you submit. In your interview, we will ask you to walk through specific choices.

Part 1 — Debug an order processing function

Background

Rockwell Razors runs a custom integration system that processes e-commerce orders. When a customer places an order on Shopify, it flows through an AWS-based pipeline: Shopify → webhook → SQS → Lambda → DynamoDB → ERP system.

We've extracted a simplified version of one of our real order processing Lambda functions. It:

1. Receives an order payload from an SQS queue
2. Validates the order data
3. Looks up product mappings from the ERP API
4. Creates a sales order in the ERP
5. Updates the order status in DynamoDB

The function has 3 bugs. Some are obvious. Some are subtle. All of them would cause real problems in production.

What we give you

A starter repo — `part-1-starter.zip`, included in the shared folder alongside this brief. The folder structure:

```
part-1/
├─ src/
│  ├─ handler.ts          ← The buggy Lambda handler
│  ├─ services/
│  │  ├─ dynamo.ts       ← DynamoDB operations
│  │  ├─ erp-client.ts    ← ERP API client
│  │  └─ validator.ts     ← Order validation
│  └─ types.ts            ← Type definitions
├─ tests/
│  ├─ handler.test.ts     ← Test suite (some will fail)
│  └─ fixtures/           ← Test order payloads
├─ docker-compose.yml     ← Local DynamoDB + mock ERP API
├─ package.json
└─ tsconfig.json
```

Setup

You'll need **Node.js 18+** and **Docker Desktop** installed.

```
# 1. Unzip part-1-starter.zip and cd into the folder

# 2. Install dependencies
npm install

# 3. Start local DynamoDB + mock ERP API
docker compose up -d

# 4. Run the test suite
npm test
```

Several tests will fail out of the box — that's expected. The bugs are causing them to fail.

Your assignment

1. **Find and fix all 3 bugs.**
2. **Make all tests pass.**
3. **For each bug, add a 2–3 sentence comment above your fix** explaining:
 - What the bug was
 - What it would cause in production
 - How you found it (reading the code, running tests, AI flagged it, etc.)

How we evaluate Part 1

Criteria	What we look for
Bug detection	Did you find all 3? Did you explain them clearly?
Fix quality	Are the fixes correct and clean — not band-aids?
Debugging process	Your comments tell us how you think. Did you actually read the code, or just throw AI at it?
Can you run it?	Did you get Docker + local DynamoDB + tests working?

Part 2 — Build a UI component with Claude Code CLI and Figma MCP

This is the part of the test we weight most heavily. It mirrors exactly how you will spend most of your time in this role.

Background

We ship small, well-scoped UI components directly from Figma designs using an AI-first workflow: Claude Code CLI (or Codex) for the code, Figma MCP for pulling spacing, colors, typography, and assets out of the design file.

What we give you

- A starter repo — `part-2-starter.zip`, included in the shared folder alongside this brief
- A Figma file with the component design: [Open the Figma file](#)

The starter is built on our actual production stack:

Layer	Technology
Framework	React 19 + Vite
Language	TypeScript
Styling	Tailwind CSS v3
UI primitives	shadcn/ui (new-york style, slate base)
State	Zustand

`Button` and `Card` are pre-installed. You can add more shadcn components as needed with `npx shadcn@latest add <component>`. Path alias `@/*` is wired to `src/*`.

Setup

```
# 1. Unzip part-2-starter.zip and cd into the folder
```

```
# 2. Install dependencies
```

```
npm install
```

```
# 3. Run the dev server
```

```
npm run dev
```

Open the URL Vite prints (usually `http://localhost:5173`). You should see a working demo card with a Zustand counter — that confirms the stack is wired up correctly.

Your assignment

1. **Delete the demo App contents.** It's there to prove the stack works, not as a reference design.
2. **Open the Figma file** (also linked above).
3. **Using Claude Code CLI (or Codex) with Figma MCP**, implement the component from the Figma design with 1:1 visual fidelity. Using MCP is required — we are evaluating your AI + MCP workflow, not just the finished component.
4. Use Zustand for any component state that has real state (toggles, filters, selections). Use React local state for purely visual state (hover, focus).

Deliverables

1. **Your implemented component, pushed to a GitHub repo with a clean, atomic commit history.**
 - We read your commits. Sloppy commit hygiene is a signal.
2. **A screen recording (10–15 minutes, mandatory).**
 - Loom preferred, QuickTime acceptable.
 - Pick your most representative 10–15 minute segment from your ~1.5 hour build. You do **not** need to record the entire session.
 - Narrate as you go: what you're prompting, how you're using Figma MCP, how you're iterating, how you're verifying AI output.
 - **Do not edit or polish the recording** beyond trimming to the representative segment. We want to see how you actually work.
3. **A `RESPONSES.md` file in the Part 2 repo — 150 words max.**
 - What worked in your AI workflow
 - What didn't
 - What you'd do differently next time

How we evaluate Part 2

Criteria	What we look for
Prompting quality	Scoped, specific prompts — not shotgun “build it all” asks
Iteration speed	Figma → working component, how fast and how cleanly
MCP fluency	Do you read MCP output and act on it, or stare at it?
Verification discipline	Do you trust AI output blindly or actually check the diff?
Commit discipline	Atomic commits, meaningful messages
Design fidelity	Does the rendered component match the Figma?

The recording is where we catch candidates who look polished on paper but actually fight their tools.

Submission

Reply to this Indeed message with:

1. **Link to your Part 1 GitHub repo** — bug fixes + comments
2. **Link to your Part 2 GitHub repo** — implemented component + `RESPONSES.md`
3. **Link to your Part 2 screen recording** — Loom or shared QuickTime file
4. **A short top-level note** (2–3 sentences) saying which AI tools you used and anything we should know before reviewing

Repos can be public or private. If private, please share a link that gives us read access (for example, invite the GitHub account we provide when we reply).

Rules and general guidelines

- **AI tools are encouraged throughout.** Please tell us which tools you used and how.
- **You must understand and be able to explain every line of code you submit.** In your interview, we will ask you to walk through specific sections. If AI generated something you can’t explain, that’s worse than writing it yourself.
- **Don’t over-engineer.** Don’t build a framework when a component will do. We want to see taste and judgment, not production-grade abstractions on a short test.
- **Don’t hide your AI usage.** We genuinely want to see it.
- **Don’t edit your Part 2 recording** beyond trimming to the representative segment. No voiceover polish, no cut scenes.
- **Use only free resources and tools. Rockwell will not cover any paid subscriptions.** Free tiers of Claude Code CLI, Codex, Figma, Loom, and GitHub are all sufficient for this test. If you already have a paid subscription, you are welcome to use it, but it is not required.

Good luck — we're looking forward to seeing how you work.

— Rockwell Razors Engineering